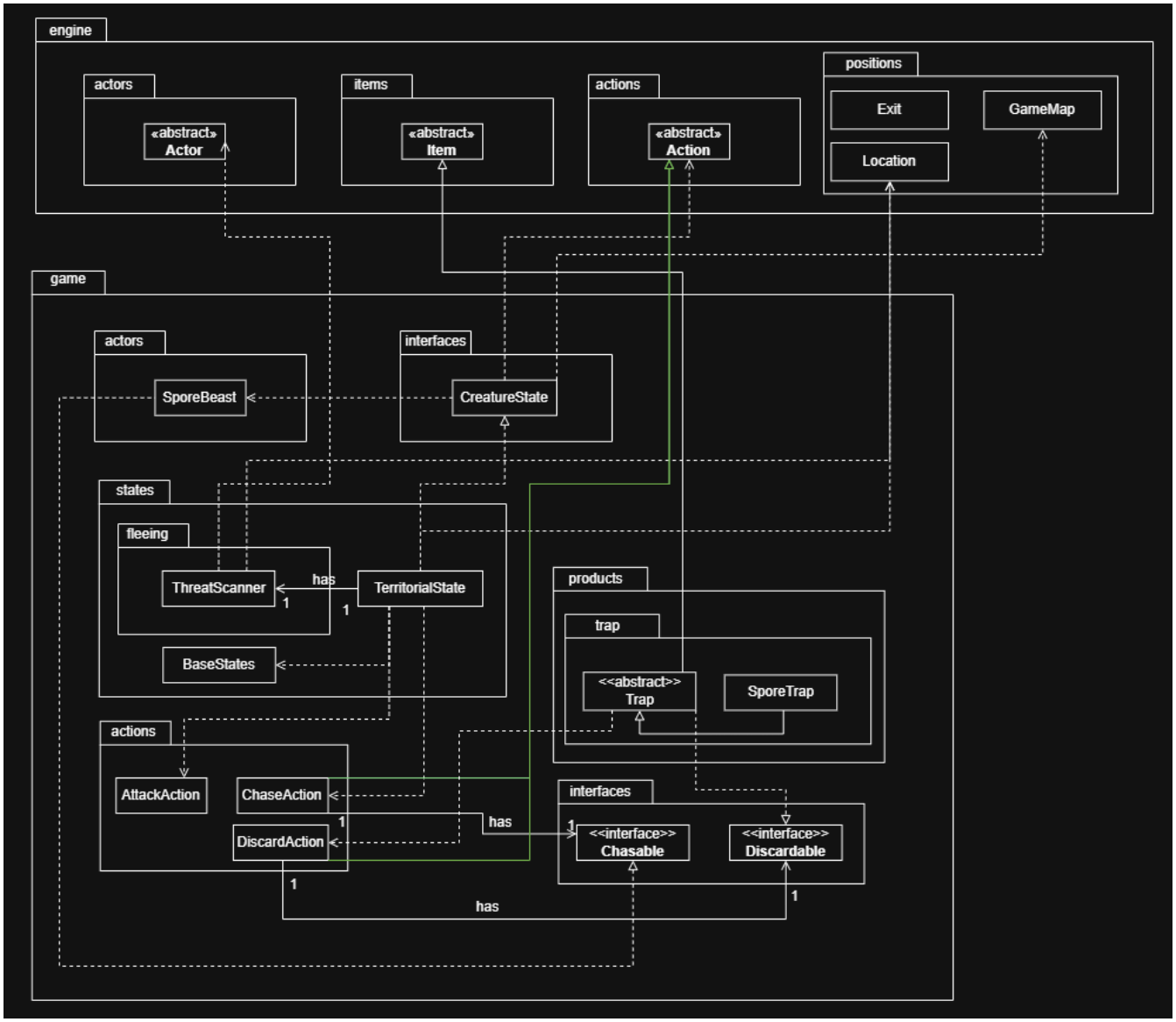
REQ5 The Territorial State

Design Diagram

REQ5 Territorial State UML Class Diagram



A Creation of Territorial State

Design 1 Not Selected:

Create concrete class of Territorial State without extends from any super class or implements any interface, construct own methods with same parameters as the other states.

Advantages	Trade-offs
- Simple & straightforward, as territorial states only contain the effects and actions of current state.	- Increase code redundancy, as the method name and signatures are the same, without any extends or implements, the same method name will be repeatedly occurred. Bad design. - Increase dependency and coupling, since there's no implementation of interface, as parameters in method signature increase, dependency increase.

Design 2 Selected:

Create concrete class of Territorial State that implement the existing CreatureState class, override the necessary methods and construct own implementation.

Advantages	Trade-offs
- TerritorialState encapsulates everything about the territorial behaviour (chasing, attacking, deciding when to leave the state) in one class, so the Sporebeast itself doesn't hold that logic. - New states (e.g. a fourth or fifth state) are added as new CreatureState implementations, without modifying the Sporebeast or existing states. - The Sporebeast can hold any CreatureState and treat TerritorialState like any other, as long as the overridden methods honour the interface's contract (valid actions, no unexpected exceptions or null).	- If CreatureState is wide (e.g. enter, exit, action, transition, description), TerritorialState may be forced to implement methods it only stubs out, which is a sign the interface is too large.

B The Discardable Trap & Spore Trap

Design 1 Not Selected:

Create a concrete class for trap without extend from the Item super class from engine package, rewrite all the entities and behaviors of a trap inside the Trap class. For a spore trap, create a new method to hold the spore effect. Create an interface Discardable holding the method to discard a trap and create a concrete class DiscardAction directly hold the trap.

Advantages	Trade-offs
- Discardable has one job (the discard contract) and DiscardAction only hold discarding. Spore-specific logic sits in its own method rather than being scattered.	- A Trap is not an Item, so it can't be placed on a map, held in an inventory or picked up like other items, and it can't be passed to engine code that expects an Item. The engine can't substitute it anywhere. - Adding a new trap type (e.g. a fire trap) means editing Trap to add another effect method, or branching on the type. Every new effect changes an existing class. - Rewriting the Item entities and behaviours duplicates engine code. Any bug fix or engine change to Item must be mirrored in Trap, and every other item-like class may repeat the same work.

Design 2 Selected:

Create a abstract class for trap extends from the Item super class from engine package, override the entities and behaviors of a trap inside the Trap class. For a spore trap, create an abstract method to hold the additional effect in Trap class, SporeTrap extends from super class Trap and override the additional effect method. Create an interface Discardable holding the method to discard a trap and create a concrete class DiscardAction directly hold the discardable trap.

Advantages	Trade-offs
- A new trap type (e.g. FireTrap) extends Trap and implements the additional effect method, with no change to Trap, SporeTrap or DiscardAction. This is a clear improvement over the standalone version. - Engine code works with Item and Action abstractions. DiscardAction depends on Discardable rather than on a specific trap class. - Shared trap behaviour is written once in Trap, and Item provides the engine plumbing, so nothing is duplicated. Each trap type adds only its own effect.	- The abstract method forces every Trap subclass to implement the additional effect, even a plain trap with no extra effect, which then needs an empty override.

C Chasing of the Spore Beast

Design 1 Not Selected:

Create concrete class for chase action with no inheritance from any super class, stores all the entities and methods inside the ChaseAction class. For the actor that can perform the chase action, direct execute the chase action effect inside the actor class.

Advantages	Trade-offs
- ChaseAction has one clear purpose (chasing), and Chasable only declares the capability to chase. The Sporebeast doesn't need to hold chase logic in its own class.	- Without the engine's Action type, ChaseAction can't be substituted wherever the engine expects an action. It can't be returned from allowableActions or playTurn directly, and the engine can't execute it through its normal flow. - High-level engine code (the creature's turn logic, state classes such as TerritorialState) must depend on the concrete ChaseAction, since there is no abstraction to depend on. - ChaseAction directly depends on the actor instead of an abstract contract, increase dependency.

Design 2 Selected:

Create concrete class for chase action inherit from Action super class, override the needed entities and methods inside the ChaseAction class. For the actor that can perform the chase action, create an interface Chasable maintaining the chase method, let the SporeBeast class implement this interface.

Advantages	Trade-offs
- The engine depends on the Action abstraction, and ChaseAction depends on Chasable rather than on Sporebeast, so it works with any chaser. - Chasable has a single method, so implementers aren't forced to depend on anything unused. Only actors that can chase implement it. - ChaseAction honours the Action contract, so the engine, menus and states can use it like any other action. Any Chasable actor can be used by it. - New chasing actors implement Chasable with no change to ChaseAction. New actions extend Action without touching the engine or existing actions.	- Variations of the chase (e.g. a faster chase, one that gives up after a distance) require editing ChaseAction or subclassing it, since the algorithm is fixed inside.

=

Conclusion

Overall, once the spore beast get into the territorial state, depends on the number of actors or enemies around, it will fall into three actions, chase the enemy, attack the enemy, or do nothing. Spore beast attacks the actor via their intrinsic weapons, reduce the health of the actor. Once enemies are detected, spore beast will spread spore traps in its surrounding, the player should have some tools to discard the trap, or negative effects will lay on the player. If the player is out of the attackable range, it turns to chasing the player in its chasable range. When there are more actors around, the spore beast will fall into other states.

For implementation A, design 2 is selected because instead of directly referencing the territorial state object, adding an abstract contract can reduce the class coupling and dependency (Dependency Inversion Principle). Especially when several states containing same methods, but different implementations, interface will be a better choice than concrete or abstract class, for better extensibility (Open-Closed Principle). Since territorial state should only managed the effects when reached to the current state, create concrete class to implement interface to take over the effects (Single Responsibility Principle), or responsibility will be overlapped across classes.

For implementation B, design 2 is selected to follow Liskov Substitution Principle, as any changes in the Trap class may affect the operations in DiscardAction class. The Discardable interface is holding the agreed contract between high- and low-level module, as long as the interface remain, the changes in modules won't affect the operation. For inheritance of the Trap from Item class, it is both to reduce code redundancy for better extensibility, like override the methods and implement own operations (Open-Closed Principle), and obey to Liskov Substitution Principle, where the trap can act as an Item to perform the Item behaviors or upcasting safely when needed.

For implementation C, design 2 is selected, since in the main operational class of this application, Action super class is used as an abstract contract to reference to different kinds of action with specific implementations, inheritance supports polymorphism (Liskov Substitution Principle). Like when calling for ChaseAction object, the operational class will be aware that exist an execute method, since the Action class already contains it. The created Chasable interface is not only align with Dependency Inversion Principle, where high-level modules won't directly reference to the low-level modules; but also separating the responsibilities of the interfaces, avoid implement an interface with unused method (Interface Segregation Principle).

All in all, although the selected designs obey to most of the design principles, but the trade-offs should not be ignored, further improvement and code refactoring is required.